

批改单 · D8 作业

文件 我的练习/D8作业.py (81 行) 批改于 2026.09.28 (周一) 结论 ● 需返工 —— 4 / 5 项验收未过

下面每一条结论都**真跑过**，不是看代码猜的。我把你作业里每一处可疑点单独拿出来跑了一遍， 原始输出留在 `我的练习/_批改-D8-输出.txt`（脚本 `_批改-D8.py`，你可以自己再跑一次）。

你在这里 > [阶段 A](#) > [② 编程主线 · SQL](#) > [第 3 课 · D8](#) 复习台 函数卡 第 3 课讲义

判 先看判决：五项验收

要求	判	实测到什么
三张 <code>CREATE TABLE</code>	半过	三张都建出来了，方向也对；但里面 3 条 REFERENCES 没有一条真的成立
插 3 行数据	未过	库里 0 行 —— 程序崩了，数据全被回滚（下面 §2）
一条 <code>JOIN</code> 跑通	未过	一条 JOIN 都没有 —— 三次查询都是单表 <code>SELECT *</code>
坑一： <code>ON</code> 故意接反	未过	没做
坑二：数行数 + 核对一行	未过	没做 —— 但你的 bug 已经替你生成了这道题的答案 (§6)

说清楚：**这不是“全错”**。三张表的结构、绳子表的存在、`Path(__file__)` 的用法都是对的 (§1)。问题集中在两处 —— **一股脑的笔误** (§3) 和 **一个设计判断** (§5)。后者才是这次要学的东西。

对 做对的四件事（别改掉）

- `Path(__file__).with_name("D8.db")` —— 这正是上周 `pytest` 那个坑的修法：用脚本自己的位置拼路径，**从哪个目录运行都不炸**。我特意在 `C:\Users\vision` 下跑你的脚本，它照样找到了 `D8.db`。✅ 这一条比作业本身值钱。
- `DROP TABLE IF EXISTS` —— 脚本可以反复跑，不用手删库。✅
- 占位符** `?` + `executemany` —— 没有把值拼进 SQL 字符串。✅ 习惯对了。
- 三张表的选型** —— `students` / `course` / `student_course`，第三张就是绳子表。**你确实推出了“要第三张表”**，这一步是第 3 课的核心，你拿到了。

空 最要紧的一件事：你的库现在是空的

我打开 `D8.db` 看到的是：**三张表都在，但加起来 0 行数据**。如果你之前觉得"表建好了就是跑通了"，那是假象。

```
实测 — 直接读你交上来的 D8.db

students      -> 0 行 []
course        -> 0 行 []
student_course -> 0 行 []
```

为什么表在、数据没了？ 实测 `conn.in_transaction`：

```
实测 — 探针 0

刚连上          in_transaction = False
执行 DDL(CREATE) in_transaction = False  <- 建表不进事务，直接落盘
执行 DML(INSERT) in_transaction = True   <- 插数据才开事务
崩溃后重连 — 表还在吗： ['t']
崩溃后重连 — 数据还有吗： []
```

`CREATE TABLE` 是**立刻生效**的，所以表留下了； `INSERT` 开了一个事务，而脚本在第 62 行崩了、**没人 `commit`**，连接被回收时整个事务回滚 ⇒ **连你插的 5 个学生、6 门课也一起没了**。

记住这条：**程序崩了 = 从上次 `commit` 之后做的事全部作废**。你的 `commit()` 写在 `link()` 里，所以只要第一次调用 `link()` 就炸，前面所有插入都白干。这不是运气问题 —— 是"commit 放哪"的问题。

崩 报错链：修一个撞下一个（4 处）

你现在停在第一个。我把后面三个也提前跑出来了 —— 免得你修一个跑一次、跑一次撞一个。

行	你写的	实测报错
44	<code>cur.execute('... WHERE name = ?', course_name)</code>	<code>ProgrammingError: Incorrect number of bindings supplied. The current statement uses 1, and there are 2 supplied.</code>
52	<code>INSERT INTO student_course(..., class_naem)</code>	<code>OperationalError: table student_course has no column named class_naem</code>
75	<code>SELECT * FROM courses</code>	<code>OperationalError: no such table: courses</code>
79	<code>SELECT * FROM student_courses</code>	<code>OperationalError: no such table: student_courses</code>

第 44 行那个逗号，值得单独讲 —— 它会"时好时坏"

你写的是 `course_name`，少了一个逗号，所以传进去的**不是一个元组，是一个字符串**。而 `sqlite3` 把字符串当**一串参数**看： `'语文'` 有 **2 个字符** ⇒ 它以为你给了 2 个参数 ⇒ 报错。

实测 — 探针：换个课名会怎样

'语文' 长度 2 -> `ProgrammingError Incorrect number of bindings supplied. ... there are 2 supplied.`

'语' 长度 1 -> 静默通过，返回 `[('语',)]`

'数学' 长度 2 -> `ProgrammingError Incorrect number of bindings supplied.`

看第二行：如果课名是 1 个字，这个 bug 会安静地跑过去。这类“随数据变化忽隐忽现”的错最难查——所以正确的写法是永远写成元组：`(course_name,)`。那个逗号不是排版，它是语法。

顺带：`courses` / `student_courses` 在表里叫 `course` / `student_course`。建表、外键、查询三处用的名字要一模一样——这类错 SQLite 从不帮你猜。

哑 不报错、但数据是错的（两处，比上面危险）

① 列名和值没对齐——静默存反

-- 第 52-53 行，你的写法：

```
INSERT INTO student_course(course_id, student_id, class_name) VALUES (?, ?, ?)
                        ↑ 顺序 A                                (sid, course_id, class_name)
                        ↑ 顺序 B — 反了
```

两个都是整数，SQLite 照单全收，**不报错、不警告**。跑出来的后果（实测，把你那 5 行数据按原样灌进去）：

实测 — 探针 3：绳子表 5 行，配一条正确的 JOIN

你本来想记的：张三 选了 物理

绳子表里实际存的：`(course_id=1, student_id=4, '高一')`

↓ 用 `student_id=4` 去 JOIN `students`

JOIN 读出来的： 李三 选了 语文 ← 人错了、课也错了

实测 — 5 行跑完的完整结果

绳子表 `student_course` 全部行：

<code>(1, 4, '高一')</code>	<code>(2, 1, '高三')</code>	<code>(3, 6, '高三')</code>
<code>(4, 3, '高一')</code>	<code>(5, 5, '高二')</code>	

JOIN 出来：

<code>(1, '李三', '语文', '高一')</code>	
<code>(2, '张三', '数学', '高三')</code>	← 张三其实是高一
<code>(4, '牛二', '物理', '高一')</code>	← 牛二其实是高三
<code>(5, '张贾瑞', '化学', '高二')</code>	

绳子表 5 行 → JOIN 只得 4 行

绳子表里指向不存在学生的行：`[(6,)]`

报错了吗？没有。行数对吗？不对（5→4）。内容对吗？错。这就是讲义里说的"看着合理的假数据"——而且是你自己制造的第一个真实样本。

顺便你会发现：`class_name` 这一列**自己就是破案线索**。JOIN 出来「张三·高三」，可名册里张三明明是高一的——两份数据自己打起来了。下次不知道该核对什么的时候，就找这种"应该一致的地方"。

② 三条 `REFERENCES`，三种不同的错法

昨天你刚问过 `REFERENCES students(id)` 是什么。这里正好是它的**应用题**——而你三条全踩了不同的坑。实测：

你写的	问题	实测
<code>REFERENCES courses(id)</code> 第 27 行	父表叫 <code>course</code> ，不叫 <code>courses</code>	建表：成功 插入：no such table: main.courses
<code>REFERENCES students(class_name)</code> 第 29 行	父键 不是 <code>UNIQUE</code> （一堆人都是高一，指向哪一行？）	插入：foreign key mismatch - "student_course" referencing "students"
<code>REFERENCES students(id)</code> 第 28 行	✅ 写法对——但 默认不生效 （pragma 没开）	pragma 开：插 999 号学生 → <code>FOREIGN KEY constraint failed</code> pragma 关：静默插进去

建表的时候这三条 `SQLite` 一条都不检查——全部“成功”。你会以为写对了。等真往里插数据才炸，而且炸的是最难懂的那种错。⇒ 这正好是函数卡 `sql.foreign_key` 那张的实战版，去搜「外键」重看一遍。

实测 — 对照：给 `class_name` 加 `UNIQUE` 之后，第 29 行就能生效了
插入 高一：成功
插入 高二：`IntegrityError -> FOREIGN KEY constraint failed`

设 这次最关键的一条：`class_name` 不该待在绳子表里

你在绳子表里加了第三列 `class_name`，还给它写了外键。看格式，你大概是照着你自己那个处方库的样子写的——那个库里，绳子表确实有第三列：

```
-- 你的 prescriptions.db, prescription_ingredients 表
CREATE TABLE prescription_ingredients (
  prescription_id INTEGER,
  ingredient_id   INTEGER,
  dosage          TEXT,      -- ← 绳子表里的第三列，你是照这个抄的
  ...)
```

所以区别在哪？一句话：

- `dosage`（三两）是在说这次配对本身 —— 当归在三两这个方子里是三两，换个方子就不是了。⇒ 属于绳子表 ✓
- `class_name`（高一）是在说这个人 —— 张三在不在语文课上，他都是高一。⇒ 不属于绳子表 ✗

判据：把配对拆开，那一列还成立吗？成立 → 它不属于绳子表。

实测代价：改一次名册，两份数据就对不上了

实测 — 探针 5

分班时：`students` 说 高一 / `sc` 说 高一

张三升到高二，只改了名册这一处（这是唯一正确的改法）：

`students` 说 高二 / `sc` 说 高一 <- 假的

而且 `SQL` 不会告诉你哪一份对：两份都是普通文本，谁也不指向谁

对照：绳子表只留两列（`course_id`, `student_id`），班级要用时现 `JOIN` 出来
名册改完，`JOIN` 出来立刻是新的：`[( '张三', '高二', '语文' )]`

这叫改一处、漏一处。你现在有 5 个学生，手工同步还行；等有 500 行，库里就会同时存在"高一"和"高二"的张三，而没有任何东西会报错。

还有更要紧的一层：你把班级抄进绳子表，动机很可能是"这样就不用 `JOIN` 了" —— 一句 `SELECT *` 就能看到全部信息。

但这次作业要交的恰恰就是那条 `JOIN`。关系模型的做法是：名字不抄、`id` 只存一次，需要的时候现配。抄一份 = 把 `JOIN` 的活儿提前干了一遍，还干错了（见上面）。

要你回答（一句话，别展开）：

- 为什么 `dosage` 该在绳子表、`class_name` 不该？
- 如果以后真要记"这个学生在这门课上的成绩"，成绩该放哪张表？为什么？

查 自查清单：5 项，你写出来 0 项

但其中一项，你的 bug 已经替你答了：

绳子表是哪一张？主键是什么？

你写了 `PRIMARY KEY (course_id, student_id, class_name)` ——多塞了一列。实测：同一门课重复插，`UNIQUE constraint failed`（碰巧防住了，因为班级不会变）；但把 `class_name` 写成别的值，同一个人就能重复选同一门课了（实测：插入成功）。正确的主键是前两列。

一条 JOIN 跑出来几行？由哪张表决定？

你的数据已经给出了答案：绳子表 5 行 → JOIN 出来 4 行。少的那行是 `(3, 6, '高三')` —— 6 号学生根本不存在，JOIN 配不上就直接丢掉（INNER JOIN 的行为）。行数由绳子表决定，但能被配上的行才留下。

把 `ON` 接反再跑一次 —— 0 行还是假数据？

没做（\$8 第 4 步）

挑一行人工核对内容

没做（上面那张表就是现成的素材：核对 `class_name` 和 JOIN 出来的学生对不对得上）

接反之后为什么没报错？

没答。提示：SQL 只比较“两个数相等吗”，它不知道这两个数本来是什么意思。

死 第 58–68 行：那段 if/else 是死代码

```
b = -1                                # ← 循环外面（没用到）
for s in student:
    rn = rnd.randint(0, 5)
    b = -1                            # ← 问题在这：每轮开头又把它按回 -1
    if rn != b:                       # rn 是 0-5, b 是 -1 ⇒ 永远为真
        link(s[0], course[rn][0])
        b = rn                      # ← 白写：下一轮开头就被覆盖
    else:
        ...                          # ← 永远走不到
```

实测 — 探针 4

`rn` 从 0 到 5 各跑一次，`if` 分支进入次数 = 6 `else` 分支进入次数 = 0
⇒ `else` 分支是死代码，永远走不到；`b = rn` 这行也没用（下一轮开头又被 `-1` 覆盖）

你八成是想做“这个人已经选过这门课了，就换一门”。但 `b` 在循环里被重置，所以那个判断从来没生效过 —— 而它长得像在工作，这就是死代码最坑的地方。（顺带：`else` 里那句 `course[rn+1]` 在 `rn=5` 时会越界，只是因为走不到才没爆。）

我看到你旁边还有个 `测试.py`，里面就是 `if a != b:` 然后空着 —— 实测跑它：`IndentationError: expected an indented block after 'if' statement on line 5`。那个文件从头到尾没跑起来过。下次把试的那两行直接写在作业脚本里试，别另开一个不跑的文件。

还有一件跟作业验收直接相关的：`rnd.randint` 让每次跑出来的行数都不一样。讲义要求“挑一行人工核对内容”，随机数据没法核对、也没法复现。作业里的数据写死更好 —— 随机是后面写压力测试才需要的东西。

返工顺序（约 30–40 分钟）

按这个顺序做，中间不要跳 —— 前两步是清障，第三步才是这次真正要学的。

第 1 步：清掉 4 个笔误（5 分钟）

- ☐ 第 44 行 `course_name` → `(course_name,)` —— 补逗号
- ☐ 第 52 行 `class_naem` → 拼对
- ☐ 表名统一：`courses` → `course`（第 27 / 75 行）、`student_courses` → `student_course`（第 79 行）

第 2 步：把列名和值对齐（3 分钟）

- ☐ 第 52–53 行：列写什么顺序，值就给什么顺序。写完把两行并排看一遍再回车。

小技巧：列名和值竖着写成两行对齐，反了一眼就看得出来 —— 后面写大段 INSERT 都这么干。

第 3 步：绳子表砍成两列（10 分钟，这是重点）

- ☐ `student_course` 只留 `course_id` / `student_id`，主键 `PRIMARY KEY (course_id, student_id)`。`class_name` 整列删掉。
- ☐ `link()` 里那个“再查一次班级”的 `SELECT` 也一起删 —— 不需要了。
- ☐ 三条 `REFERENCES` 重写：只留指向两张主表 `id` 的两条（父表名写对）。顺便把 `PRAGMA foreign_keys = ON` 加上，让它们真的管事。

第 4 步：写那条 JOIN（10 分钟）

- ☐ 从绳子表出发，把两边的名字配出来。骨架和昨天那张单子上的一样，空得你自己填：

```
SELECT _____, _____, _____      -- 你想要哪几列？（学生名 / 班级 / 课程名）
FROM student_course
JOIN _____ ON _____ = _____      -- 配学生
JOIN _____ ON _____ = _____;      -- 配课程
```

第 5 步：补上两坑（10 分钟，不许省）

- ☐ **坑一**：把你 JOIN 里某一句 `ON` 的两头故意接反，跑一次。记下：**0 行**，还是**假数据**？（你在这套数据里 `course.id` 是 1-6、`students.id` 是 1-5，**会重叠**——所以先猜猜是哪种，再跑。）
- ☐ **坑二**：单独 `SELECT COUNT(*)` 数绳子表几行，再数 JOIN 几行，**挑一行**把学生名、班级、课程名逐个核对一遍。

第 6 步：把数据写死，再跑一遍

- ☐ 把 `rnd.randint` 换成一串写死的选课（比如 `[( '张三', '语文' ), ( '李四', '数学' ), ... ]`），这样别人（和我）拿到你的脚本，跑出来**行行一样**。

交 交回来时给我四样

- 改好的 `D8作业.py`（还是这个路径就行）
- JOIN 的真实输出**贴给我（几行、每行内容）
- 坑一的答案：接反之后是 **0 行** 还是 **假数据**？
- 两个数：**绳子表几行 / JOIN 几行**，以及你核对了哪一行

\$5 那两个问题（`dosage` 为什么该在绳子表里 / 成绩该放哪张表）也一起答了——答完这次作业就算真正结掉，第 3 课只剩 ⑤⑥（换皮 + 你讲我听），那两个**要我在场**，你答完就来找我约时间。

这张批改单是**新格式的第一次**。以后你每次交作业我都出这么一张（可打印、带实测输出、留返工清单）——觉得哪里别扭跟我说，我改。

批改单 · D8 作业 · 2026.09.28 | 世界：米黄方格纸 · 实验记录本

证据： `我的练习/_批改-D8-输出.txt`（脚本 `_批改-D8.py`，5 组探针，可重跑） | 判据： `每周书/W39-数据怎么连起来.html` 第 8 章 · `每周目标.md`（W40 第 1 项） | 配套：函数卡 · 复习台 · 今日清单